Collaboration, Quality, Test
WS2025/2026
Prof. Dr. Jóakim von Kistowski

Exercise 6 – Sudoku

Learning objectives: Create a full-scale project with automated tests

Today, we create the basis for a Sudoku game. This game will be an ongoing project over the next few exercises. We will extend and test it in various ways.

The main parts of this exercise are:

1. Create a new project with test frameworks setup
2. Create a class to check if a Sudoku is solvable
3. Create a class to set up an initial and solvable Sudoku board
4. Write Unit and Integration Tests

Sudoku is a game where a player has to complete an initially partially completed 9x9 field with numbers between 1 – 9.
Numbers must be chosen so that:

- Each number appears at most once per row
- Each number appears at most once per column
- Each number appears at most once per region (a region is a 3x3 subgrid as shown on the right)

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Because this class is mostly about testing, code snippets for the actual implementation are provided in the Moodle course. This should help you to get started quickly.

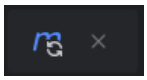
Part 1 – Create a Project

In IntelliJ:

1. Create a new Java project as done in exercise 2. This time, name it “sudoku”
2. In the pom.xml file enter all dependencies used so far in exercises 2 and 5. The dependency section should now look like this:

```
<dependencies>
  <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter-engine</artifactId>
        <version>5.14.1</version>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter-params</artifactId>
        <version>5.14.1</version>
        <scope>test</scope>
    </dependency>
  <dependency>
      <groupId>org.mockito</groupId>
      <artifactId>mockito-core</artifactId>
      <version>5.21.0</version>
      <scope>test</scope>
  </dependency>
  <dependency>
      <groupId>org.mockito</groupId>
      <artifactId>mockito-junit-jupiter</artifactId>
      <version>5.21.0</version>
      <scope>test</scope>
  </dependency>
</dependencies>
```

3. Click on the refresh button that appears to the top right or use the keyboard shortcut ctrl+shift+O



Part 2 – Implement a Class to Check if a Sudoku is solvable

Don't forget to implement unit tests for all public and package-private methods you create in this class.

In this exercise, a Sudoku board is simply a two-dimensional array of Integers with a fixed size of 9x9. The first dimension of the array represents the board's columns (**x** dimension) and the second the rows (**y** dimension).

1. Create a Java package with the name *de.thab.cqt.sudoku* in the folder *src/main/java*. We will place all of our classes here
2. Create the same Java package in the folder *src/test/java*
3. Create a class with the name *SudokuSolver* in the *de.thab.cqt.sudoku* package in *src/main/java*
4. Create a test class with the name *SudokuSolverTest* in the *de.thab.cqt.sudoku* package in *src/test/java*
5. In *SudokuSolver* implement the following method (you can find it in the provided code snippet):
   `boolean isValidSudokuBoard(int[][] board)`

   This method checks if a boards dimensions are exactly 9x9. If it is, it returns `true`, otherwise it returns `false`.
6. In *SudokuSolver* implement the following method (you can find it in the provided code snippet):
   `public boolean isSolvable(int[][] board)`
   The method should check if a Sudoku is solvable. If it is, it returns `true`, otherwise it returns `false`.
7. In *SudokuSolver* implement the following method (you can find it in the provided code snippet):
   `public boolean isSolved(int[][] board)`
   The method should check if a Sudoku is solvable and contains no empty field (a field that is 0). If it is solvable, it returns `true`, otherwise it returns `false`.
8. In *SudokuSolverTest* implement unit tests for *SudokuSolver*. Discuss with your neighbors which test cases you think are the most important.
   Hint: In a later exercise, we will discuss what test coverage is. You will be asked to add tests until you hit a certain coverage level. So don't skimp out on tests here!

Part 3 – Setting Up an initial Sudoku Board

1. Create a class with the name *SudokuCreator* in the *de.thab.cqt.sudoku* package in *src/main/java*.
   The class takes an instance of *SudokuSolver* as its constructor parameter and stores it in a private class attribute.
2. Create the corresponding test class.
3. In *SudokuCreator* implement the following method (you can find it in the provided code snippet):
   ```
   public int[][] createNewBoard(int numberOfPrefilledFields)
   ```

   This method create a new Sudoku board and fills *numberOfPreFilledFields* fields in the board with random numbers, ensuring that the Sudoku remains solvable. It throws an *IllegalArgumentException* if *numberOfPreFilledFields* is greater than 10 or smaller than 1.
   Hint: Use Java's Random class to generate random numbers, then check if the Sudoku remains solvable, then try again.
4. Create tests for your *SudokuCreator*. Mock the *SudokuSolver* in your tests.

Part 4 – Writing an Integration Test

You have already created unit tests for your classes. Now it is time to write an integration test that tests both working together.

1. Create a test class with the name SudokuCreatorITTest in the *de.thab.cqt.sudoku* package in *src/test/java*.
   Note: **IT** is the abbreviation for Integration Test. We avoid writing the word Integration Test in full, because finding a test class with "*IntegrationTest*" in its name causes Maven's (our build system) behavior to change.
2. Setup your tests so that they use actual instances of both *SudokuCreator*, as well as *SudokuSolver* (no mocks).
3. Discuss with your neighbors: What tests do you need to implement here? Consider that you have already unit tested much of *SudokuCreator* and *SudokuSolver*.
4. Implement the necessary integration tests.